

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN - ĐIỆN TỬ



BÁO CÁO BÀI TẬP LỚN HỌC PHẦN KỸ THUẬT PHẦN MỀM ỨNG DỤNG

Đề tài: Xây dựng hệ thống Custom Keyboard Builder

GVHD:

Mã lớp:

**Nhóm sinh viên
thực hiện:**

1.

MSSV:

2.

MSSV:

3.

MSSV:

4.

MSSV:

5.

MSSV:

Hà Nội, 7/2026

MỤC LỤC

MỞ ĐẦU.....	7
1. Giới thiệu và lý do chọn đề tài	7
2. Mục đích và nhiệm vụ của đề tài.....	7
3. Phân công nhiệm vụ của nhóm.....	8
4. Nội dung sơ lược của báo cáo	8
CHƯƠNG 1. TỔNG QUAN HỆ THỐNG CUSTOM KEYBOARD BUILDER.....	9
1.1. Bối cảnh và vấn đề cần giải quyết.....	9
1.2. Yêu cầu về dữ liệu	9
1.3. Mục tiêu và phạm vi hệ thống	10
1.4. Tác nhân và yêu cầu chức năng.....	10
1.5. Yêu cầu phi chức năng	11
1.6. Công nghệ và môi trường phát triển.....	11
1.6.1. Ngôn ngữ C# và nền tảng .NET 10	11
1.6.2. WPF và XAML	12
1.6.3. Microsoft SQL Server Express và SSMS.....	12
1.6.4. Microsoft Visual Studio	12
1.6.5. Keyboard kit và phạm vi phần cứng.....	12
1.7. Thư viện và thành phần kỹ thuật	13
1.7.1. Microsoft.Data.SqlClient 6.1.1	13
1.7.2. LiveChartsCore.SkiaSharpView.WPF 2.0.4	13
1.7.3. MQTTnet 4.3.7.1207	13
1.7.4. Bảo mật mật khẩu và đa ngôn ngữ	13
1.8. Kiến trúc phần mềm	13
1.8.1. Mô hình Model-View-ViewModel.....	13
1.8.2. Lớp Service.....	14
1.8.3. Lớp Repository	14
1.8.4. Hạ tầng Data/Realtime và DeviceSimulator.....	14
1.8.5. Composition root và điều hướng theo vai trò.....	14
1.9. Kết luận chương	14
CHƯƠNG 2. PHÂN TÍCH THIẾT KẾ HỆ THỐNG	15
2.1. Use Case Diagram	15
2.2. Activity Diagram	15
2.3. Sequence Diagram.....	16
2.4. Entity-Relationship Diagram (ERD)	17
2.4.1. Sơ đồ ERD tổng quát.....	18
2.4.2. Sơ đồ ERD chi tiết.....	19
2.5. Đối chiếu diagram với thành phần triển khai	19
2.6. Kết luận chương	20
CHƯƠNG 3. TRIỂN KHAI VÀ KIỂM THỬ.....	21
3.1. Triển khai cơ sở dữ liệu SQL Server.....	21

3.1.1. Nhóm tài khoản và phân quyền	21
3.1.2. Nhóm danh mục linh kiện	22
3.1.3. Nhóm build và request.....	24
3.1.4. Nhóm thiết bị và kiểm tra QC	25
3.1.5. Nhóm audit và chat.....	26
3.2. Triển khai giao diện tiêu biểu	27
3.3. Kiểm thử và xác minh	33
3.4. Đánh giá mức độ hoàn thành.....	34
3.5. Kết luận chương	34
KẾT LUẬN	35
1. Mức độ hoàn thành nhiệm vụ	35
2. Mức độ phù hợp với mục đích đề tài.....	35
3. Hạn chế còn tồn tại	35
4. Hướng phát triển.....	35

DANH MỤC HÌNH VẼ

Hình 2.1. Sơ đồ Use Case tổng hợp của hệ thống Custom Keyboard Builder	15
Hình 2.2. Sơ đồ Activity tổng hợp của các luồng nghiệp vụ chính.....	16
Hình 2.3. Sơ đồ Sequence tổng hợp của các luồng tương tác chính	17
Hình 2.4. Sơ đồ ERD tổng quát của hệ thống Custom Keyboard Builder	18
Hình 2.5. Sơ đồ ERD chi tiết gồm 21 bảng và 33 khóa ngoại.....	19
Hình 3.1. Giao diện đăng nhập của ứng dụng	28
Hình 3.2. Giao diện đăng ký tài khoản Buyer	28
Hình 3.3. Giao diện Buyer Dashboard và khu vực cấu hình build.....	30
Hình 3.4. Giao diện Seller Dashboard và xử lý yêu cầu lắp ráp	30
Hình 3.5. Giao diện Admin Dashboard và quản trị hệ thống	31
Hình 3.6. Giao diện trao đổi tin nhắn theo vai trò	31
Hình 3.7. Giao diện hồ sơ và menu người dùng.....	32

DANH MỤC BẢNG BIỂU

Bảng 0.1. Phân công nhiệm vụ nhóm.....	8
Bảng 1.1. Yêu cầu về dữ liệu.....	9
Bảng 1.2. Tác nhân và mục tiêu sử dụng hệ thống.....	10
Bảng 1.3. Yêu cầu chức năng chính.....	10
Bảng 1.4. Yêu cầu phi chức năng.....	11
Bảng 2.1. Nhóm bảng trong cơ sở dữ liệu.....	17
Bảng 2.2. Đối chiếu bốn loại diagram với artifact nguồn.....	19
Bảng 3.1. Thứ tự triển khai 21 bảng theo nhóm phụ thuộc.....	21
Bảng 3.2. Đối chiếu giao diện với mã nguồn.....	27
Bảng 3.3. Kết quả build và xác minh tại thời điểm lập báo cáo.....	33
Bảng 3.4. Đánh giá mức độ hoàn thành theo mục tiêu.....	34

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

- CSDL: Cơ sở dữ liệu.
- DBML: Database Markup Language - ngôn ngữ mô tả mô hình cơ sở dữ liệu.
- ERD: Entity Relationship Diagram - sơ đồ thực thể quan hệ.
- FK: Foreign Key - khóa ngoại.
- GUI: Graphical User Interface - giao diện người dùng.
- MQTT: Message Queuing Telemetry Transport - giao thức truyền thông publish/subscribe.
- MVVM: Model-View-ViewModel - mẫu kiến trúc giao diện.
- PK: Primary Key - khóa chính.
- QC: Quality Control - kiểm tra chất lượng.
- SQL: Structured Query Language - ngôn ngữ truy vấn có cấu trúc.
- SSMS: SQL Server Management Studio.
- WPF: Windows Presentation Foundation.
- XAML: Extensible Application Markup Language.
- ADO.NET: Tập API truy cập dữ liệu của nền tảng .NET.
- IDE: Integrated Development Environment - môi trường phát triển tích hợp.
- PBKDF2: Password-Based Key Derivation Function 2 - hàm dẫn xuất khóa dùng để băm mật khẩu.

MỞ ĐẦU

1. Giới thiệu và lý do chọn đề tài

Hiện nay, nhu cầu sử dụng bàn phím cơ tự lắp ráp và cá nhân hóa ngày càng tăng, đặc biệt đối với người làm việc thường xuyên với máy tính như lập trình viên, nhân viên văn phòng, người chơi trò chơi điện tử và người dùng phổ thông. Một bộ bàn phím tùy chỉnh cho phép lựa chọn đặc tính gõ, độ ồn, bố cục và hình thức phù hợp với nhu cầu riêng.

Tuy nhiên, số lượng loại linh kiện và tiêu chuẩn tương thích khá lớn. Việc lựa chọn riêng lẻ keyboard kit, switch, keycap, stabilizer và phụ kiện có thể gây nhầm lẫn, làm tăng thời gian tìm hiểu và phát sinh cấu hình không phù hợp. Nhu cầu quản lý tập trung cấu hình, giá tại thời điểm đặt hàng và quá trình xử lý vì vậy được đặt ra.

Từ bối cảnh trên, hệ thống Custom Keyboard Builder được xây dựng để hỗ trợ tạo cấu hình bàn phím theo mô hình kit-based, gửi yêu cầu lắp ráp cho Seller, theo dõi trạng thái, lưu kết quả kiểm tra chất lượng và quản trị dữ liệu tập trung. Ứng dụng desktop WPF phục vụ ba vai trò Buyer, Seller và Admin; SQL Server được sử dụng làm nguồn dữ liệu chính.

2. Mục đích và nhiệm vụ của đề tài

Mục đích của đề tài là xây dựng một ứng dụng quản lý cấu hình và yêu cầu lắp ráp bàn phím tùy chỉnh. Hệ thống hướng tới việc giảm khó khăn khi lựa chọn linh kiện, bảo đảm dữ liệu cấu hình được lưu nhất quán, hỗ trợ phối hợp giữa Buyer và Seller, đồng thời cung cấp công cụ quản trị và truy vết cho Admin.

Để đạt được mục đích trên, các nhiệm vụ chính được xác định như sau:

- Khảo sát bài toán, xác định tác nhân, yêu cầu chức năng, yêu cầu phi chức năng và phạm vi thực hiện.
- Phân tích và thiết kế hệ thống thông qua năm sơ đồ: Use Case, Activity, Sequence, ERD tổng quát và ERD chi tiết.
- Thiết kế cơ sở dữ liệu SQL Server, các ràng buộc toàn vẹn và dữ liệu phục vụ kiểm thử.
- Triển khai WPF theo MVVM, kết hợp Service, Repository, MQTT và mô phỏng QC.
- Thực hiện build, xác minh các luồng chính, tổng hợp kết quả và hoàn thiện báo cáo.

3. Phân công nhiệm vụ của nhóm

[Bảng 0.1](#) được bố trí để bổ sung thông tin thành viên và nhiệm vụ được phân công.

Bảng 0.1. Phân công nhiệm vụ nhóm

Thành viên	Nhiệm vụ được phân công	Sản phẩm/tiêu chí hoàn thành

4. Nội dung sơ lược của báo cáo

Báo cáo gồm 35 trang, 3 chương chính, 12 hình vẽ, 11 bảng biểu và 25 khối mã nguồn:

- Chương 1 Tổng quan hệ thống: trình bày bối cảnh, mục tiêu, phạm vi, yêu cầu, công nghệ, thư viện và kiến trúc phần mềm.
- Chương 2 Phân tích thiết kế hệ thống: tập trung phân tích năm sơ đồ Use Case, Activity, Sequence, ERD tổng quát và ERD chi tiết.
- Chương 3 Triển khai và kiểm thử: trình bày cơ sở dữ liệu, giao diện và mã nguồn tiêu biểu, kết quả kiểm thử và đánh giá mức độ hoàn thành.

CHƯƠNG 1. TỔNG QUAN HỆ THỐNG CUSTOM KEYBOARD BUILDER

Chương này trình bày bối cảnh, mục tiêu, phạm vi, yêu cầu dữ liệu và yêu cầu nghiệp vụ của hệ thống. Các công nghệ, phần mềm, thư viện và kiến trúc được giới thiệu để làm cơ sở cho hoạt động phân tích thiết kế ở Chương 2.

1.1. Bối cảnh và vấn đề cần giải quyết

Một bộ bàn phím tùy chỉnh được hình thành từ nhiều nhóm linh kiện có quan hệ tương thích về công nghệ switch, kiểu mount, layout và số lượng phím. Nếu thông tin được tìm kiếm và kiểm tra thủ công, người dùng mới có thể mất nhiều thời gian hoặc tạo cấu hình không phù hợp. Hệ thống vì vậy được xây dựng để hỗ trợ lựa chọn linh kiện có kiểm soát và lưu lại cấu hình một cách nhất quán.

Mô hình hiện tại lấy keyboard kit làm nền tảng. Một kit bao gồm case, PCB, plate và các phần đi kèm; Buyer tiếp tục chọn switch, keycap, stabilizer, accessory và ghi chú mod. Bảng build_items quy định mỗi dòng chỉ tham chiếu một loại sản phẩm, còn giá snapshot được lưu để giữ nguyên giá trị cấu hình tại thời điểm request được gửi.

1.2. Yêu cầu về dữ liệu

Dữ liệu của hệ thống được lưu tập trung trên Microsoft SQL Server Express. SQL Server Management Studio (SSMS) được sử dụng để tạo schema, thực thi script, kiểm tra dữ liệu và theo dõi các ràng buộc trong quá trình phát triển.

Các nhóm dữ liệu cần quản lý được tổng hợp tại [Bảng 1.1](#).

Bảng 1.1. Yêu cầu về dữ liệu

STT	Nhóm dữ liệu	Dữ liệu cần lưu	Yêu cầu/ràng buộc
1	Tài khoản, phân quyền và Seller	Vai trò, tên đăng nhập, email, số điện thoại, mật khẩu đã băm, trạng thái tài khoản, hồ sơ Seller và đơn đăng ký Seller.	Tên đăng nhập, email và số điện thoại không được trùng. Tài khoản không hoạt động không được đăng nhập.
2	Danh mục linh kiện	Thương hiệu, layout, keyboard kit, switch, keycap, stabilizer, phụ kiện, giá và trạng thái khả dụng.	Giá không được âm; số switch yêu cầu của kit phải lớn hơn 0. Dữ liệu phải hỗ trợ lọc và kiểm tra tương thích cơ bản.
3	Cấu hình và yêu cầu lắp ráp	Build, linh kiện đã chọn, mod, số lượng, giá snapshot, Seller tiếp nhận, nội dung JSON, trạng thái và thời điểm xử lý request.	Mỗi build item chỉ tham chiếu một loại sản phẩm. Giá và cấu hình tại thời điểm gửi request phải được giữ nguyên; trạng thái chuyển đúng luồng nghiệp vụ.

STT	Nhóm dữ liệu	Dữ liệu cần lưu	Yêu cầu/ràng buộc
4	Kiểm tra chất lượng	Thiết bị kiểm tra, phiên QC, kết quả từng phím, độ trễ, độ ồn và kết luận đạt/cảnh báo/không đạt.	Request chỉ được hoàn thành khi phiên QC gần nhất có kết quả đạt hoặc cảnh báo.
5	Đoạn hội thoại	Hội thoại, tin nhắn, người gửi, thời điểm gửi, request liên quan và lịch sử thay đổi trước/sau.	Chi hỗ trợ trao đổi Buyer-Seller và Admin-Seller.

Những yêu cầu này là cơ sở để xây dựng ERD tổng quát, ERD chi tiết và các bảng SQL được trình bày ở các chương sau.

1.3. Mục tiêu và phạm vi hệ thống

Ba tác nhân chính và mục tiêu sử dụng được tổng hợp tại [Bảng 1.2](#).

Bảng 1.2. Tác nhân và mục tiêu sử dụng hệ thống

Tác nhân	Mục tiêu
Buyer	Tạo/chỉnh sửa cấu hình và đặt hàng
Seller	Tiếp nhận và xử lý request được gán.
Admin	Quản trị người dùng và dữ liệu nền.

Phạm vi thực hiện bao gồm quản lý tài khoản và vai trò, danh mục linh kiện, cấu hình build, gửi và xử lý request, đơn đăng ký Seller, chat lưu trong cơ sở dữ liệu, thống kê dashboard, audit log và kiểm tra QC bằng dữ liệu mô phỏng.

Các nội dung ngoài phạm vi hiện tại gồm thanh toán trực tuyến, quản lý vận chuyển, kết nối ESP32 hoặc trạm đo phần cứng thật và chat realtime bằng SignalR. Các nội dung này được xem là hướng mở rộng thay vì điều kiện nghiệm thu của phiên bản hiện tại.

1.4. Tác nhân và yêu cầu chức năng

Các yêu cầu chức năng cốt lõi được liệt kê tại [Bảng 1.3](#).

Bảng 1.3. Yêu cầu chức năng chính

Nhóm chức năng	Yêu cầu	Vai trò
Tài khoản	Đăng ký làm Buyer, đăng nhập bằng username/email, xem hồ sơ, đăng xuất.	Guest và Buyer
Cấu hình build	Chọn kit, switch, keycap, stabilizer, accessory, mod; xem snapshot price và lưu build.	Buyer

Nhóm chức năng	Yêu cầu	Vai trò
Request	Chọn Seller, tạo JSON snapshot, gửi và theo dõi request.	Buyer
Xử lý Request	Chuyển trạng thái Pending -> Accepted -> In_progress -> Completed hoặc Cancelled.	Seller
Quản trị hệ thống	Quản lý Buyer, Seller, đơn Seller, catalog	Admin
Chat	Chỉ cho phép Buyer-Seller và Admin-Seller; lưu tin nhắn trong SQL Server.	Buyer, Seller và Admin
QC	Tạo session, mô phỏng từng phím, đánh giá latency/noise và tổng hợp pass/warning/fail.	Seller và device layer

1.5. Yêu cầu phi chức năng

Các yêu cầu chất lượng và vận hành được trình bày tại [Bảng 1.4](#).

Bảng 1.4. Yêu cầu phi chức năng

Nhóm yêu cầu	Yêu cầu áp dụng
Tính hợp lệ	Switch technology/mount phù hợp kit; tổng số switch ít nhất bằng required_switch_quantity.
Bảo mật	Mật khẩu lưu dạng PBKDF2-SHA256 hash; user inactive không đăng nhập.
Khả dụng	Ứng dụng hoạt động theo chế độ DB-only khi MQTT broker không khả dụng.
Dễ bảo trì	Tách View, ViewModel, Service, Repository; resource theme và localization được tổ chức riêng.
Khả năng truy vết	Thao tác quản trị quan trọng được ghi audit_log; request lưu payload JSON snapshot.

1.6. Công nghệ và môi trường phát triển

Hệ thống được phát triển dưới dạng ứng dụng desktop trên Windows. Việc lựa chọn công nghệ ưu tiên khả năng tích hợp với WPF, SQL Server và hệ sinh thái .NET, đồng thời phù hợp với mô hình phân lớp được áp dụng trong mã nguồn.

1.6.1. Ngôn ngữ C# và nền tảng .NET 10

C# là ngôn ngữ lập trình hướng đối tượng có kiểu dữ liệu chặt chẽ do Microsoft phát triển. Các đặc điểm như đóng gói, kế thừa, đa hình, generic, xử lý ngoại lệ và lập trình bất đồng bộ bằng async/await phù hợp với việc tổ chức nghiệp vụ, truy cập dữ liệu và duy trì khả năng phản hồi của giao diện.

Dự án sử dụng target framework net10.0-windows của .NET 10. Nền tảng .NET cung cấp runtime, thư viện cơ sở, cơ chế quản lý bộ nhớ và hệ thống package NuGet; hậu tố windows cho phép sử dụng WPF và các API giao diện đặc thù của Windows.

1.6.2. WPF và XAML

Windows Presentation Foundation (WPF) là framework xây dựng giao diện desktop trên Windows. WPF cung cấp hệ thống control, DataGrid, style, resource, data binding và command, qua đó hỗ trợ tách phần hiển thị khỏi logic xử lý theo MVVM.

Extensible Application Markup Language (XAML) được dùng để khai báo bố cục của Login, Buyer Dashboard, Seller Dashboard, Admin Dashboard, Chat và User Menu. Code C# phía ViewModel cung cấp dữ liệu và command; binding giúp giao diện cập nhật theo trạng thái mà không cần truy cập trực tiếp cơ sở dữ liệu.

1.6.3. Microsoft SQL Server Express và SSMS

Microsoft SQL Server là hệ quản trị cơ sở dữ liệu quan hệ, chịu trách nhiệm lưu tài khoản, danh mục linh kiện, build, request, chat, audit và kết quả QC. Cấu hình hiện tại truy cập tới instance SQL Server Express KHOADZS1VN\SQLEXPRESS và cơ sở dữ liệu CustomKeyboard_Refactor; số phiên bản máy chủ cụ thể không bị khóa cứng trong mã nguồn.

SQL Server Management Studio là công cụ quản trị được sử dụng để chạy DDL, seed data, truy vấn kiểm tra và quan sát quan hệ giữa các bảng. SQL Server là nơi lưu dữ liệu chính; SSMS chỉ là công cụ làm việc với máy chủ, không phải nơi lưu dữ liệu của ứng dụng.

1.6.4. Microsoft Visual Studio

Microsoft Visual Studio là môi trường phát triển tích hợp (IDE) phục vụ soạn thảo C# và XAML, quản lý project SDK-style, khôi phục package NuGet, build, gỡ lỗi và theo dõi lỗi biên dịch. Dự án cũng có thể được build bằng lệnh dotnet khi máy đã cài .NET SDK tương ứng.

1.6.5. Keyboard kit và phạm vi phần cứng

Khái niệm keyboard kit trong đề tài là bộ sản phẩm nền gồm case, PCB, plate và các phần đi kèm; đây không phải development kit hoặc bo mạch điều khiển dùng để phát triển phần cứng. Buyer lựa chọn kit từ catalog rồi bổ sung switch, keycap, stabilizer, accessory và mod để tạo build.

Phiên bản hiện tại không sử dụng ESP32, Arduino hoặc thiết bị nhúng thật. Quá trình kiểm tra từng phím được mô phỏng bằng DeviceSimulator; hướng kết nối trạm QC vật lý chỉ được xem là khả năng mở rộng sau phạm vi đề tài.

1.7. Thư viện và thành phần kỹ thuật

Các package NuGet và thành phần kỹ thuật bổ sung được lựa chọn theo nhu cầu truy cập dữ liệu, trực quan hóa dashboard, truyền thông realtime, bảo mật và đa ngôn ngữ.

1.7.1. Microsoft.Data.SqlClient 6.1.1

Microsoft.Data.SqlClient là data provider thuộc ADO.NET dành cho SQL Server. Thư viện cung cấp SqlConnection, SqlCommand, SqlParameter, SqlDataReader và transaction. Các Repository sử dụng câu lệnh có tham số để giảm nguy cơ SQL injection và ánh xạ dữ liệu trả về thành Model của hệ thống.

1.7.2. LiveChartsCore.SkiaSharpView.WPF 2.0.4

LiveChartsCore kết hợp SkiaSharp và WPF để hiển thị biểu đồ KPI, doanh thu, trạng thái request và dữ liệu phân tích trên dashboard. Dữ liệu biểu đồ được ViewModel nhận từ StatsService thay vì được truy vấn trực tiếp trong View.

1.7.3. MQTTnet 4.3.7.1207

MQTTnet là thư viện .NET triển khai giao thức MQTT theo mô hình publish/subscribe. Trong hệ thống, MQTT được dùng tùy chọn để phát thông báo request, cập nhật trạng thái và truyền telemetry QC qua broker mặc định localhost:1883. Cơ chế được triển khai theo best-effort; khi broker không khả dụng, luồng nghiệp vụ chính vẫn tiếp tục ở chế độ DB-only.

1.7.4. Bảo mật mật khẩu và đa ngôn ngữ

Mật khẩu được xử lý bằng PBKDF2-SHA256 của thư viện mật mã tích hợp trong .NET, với 100.000 vòng lặp, salt ngẫu nhiên 16 byte và hash 32 byte. Giá trị mật khẩu rõ không được lưu trong SQL Server. Phần giao diện sử dụng resource và lớp localization nội bộ để chuyển đổi nội dung giữa tiếng Việt và tiếng Anh.

1.8. Kiến trúc phần mềm

Kiến trúc ứng dụng được tổ chức theo MVVM và mở rộng bằng các lớp Service, Repository, Data/Realtime. Cách phân chia này giúp mỗi nhóm lớp có trách nhiệm rõ ràng và hạn chế việc trộn mã giao diện, quy tắc nghiệp vụ và câu lệnh SQL trong cùng một thành phần.

1.8.1. Mô hình Model-View-ViewModel

Model nằm trong Models/**/*.cs, biểu diễn dữ liệu và đối tượng nghiệp vụ như User, KeyboardBuild, BuildRequest, linh kiện, hội thoại và kết quả QC. Model được Repository tạo từ dữ liệu SQL Server, được Service xử lý và được ViewModel sử dụng để cung cấp dữ liệu cho giao diện.

View nằm trong Views/*.xaml, chịu trách nhiệm khai báo bố cục, control, binding, command, DataGrid, biểu đồ và trạng thái hiển thị. View không trực tiếp viết câu lệnh SQL hoặc quyết định quy tắc chuyển trạng thái nghiệp vụ.

ViewModel nằm trong ViewModels/*ViewModel.cs, giữ state của màn hình, cung cấp command bất đồng bộ và điều phối Service. Khi thuộc tính trong ViewModel thay đổi, cơ chế binding thông báo cho View cập nhật nội dung tương ứng.

1.8.2. Lớp Service

Service nằm trong Services/*.cs và thực thi các trường hợp sử dụng của hệ thống. Các lớp này kiểm tra dữ liệu đầu vào, tính tương thích linh kiện, phân quyền, state machine của request, điều kiện hoàn thành QC và các quy tắc nghiệp vụ trước khi yêu cầu Repository đọc hoặc ghi dữ liệu.

1.8.3. Lớp Repository

Repository gồm các interface trong Repositories và phần hiện thực SQL Server trong Repositories/SqlServer. Repository mở kết nối, thực hiện câu lệnh SQL có tham số, quản lý transaction khi cần và ánh xạ kết quả thành Model. Nhờ đó, ViewModel và Service không phải phụ thuộc trực tiếp vào chi tiết truy vấn SQL.

1.8.4. Hạ tầng Data/Realtime và DeviceSimulator

Data/SqlServer cung cấp cấu hình, connection factory và kiểm tra schema. Realtime triển khai MQTT phục vụ thông báo và telemetry. DeviceSimulator tạo dữ liệu QC; nếu MQTT không khả dụng, kết quả vẫn được xử lý và lưu trong SQL Server.

1.8.5. Composition root và điều hướng theo vai trò

MainWindow là composition root, tức điểm lắp ráp thành phần khi ứng dụng khởi động. Tại đây, connection factory, Repository, Service, MQTT, DeviceSimulator và MainShellViewModel được khởi tạo, cấu hình và truyền phụ thuộc; MainWindow không thực thi nghiệp vụ thay Service.

Sau đăng nhập, MainShellViewModel kiểm tra role và điều hướng tới dashboard Buyer, Seller hoặc Admin. Luồng chính đi theo View -> ViewModel -> Service -> Repository -> SQL Server; MQTT chỉ bổ sung thông báo gần thời gian thực.

1.9. Kết luận chương

Chương 1 đã xác định bối cảnh, mục tiêu, phạm vi, yêu cầu, công nghệ, thư viện và kiến trúc của hệ thống. Trên cơ sở đó, Chương 2 mô hình hóa các yêu cầu bằng năm sơ đồ thiết kế.

CHƯƠNG 2. PHÂN TÍCH THIẾT KẾ HỆ THỐNG

Chương này tập trung phân tích và thiết kế hệ thống thông qua năm sơ đồ: Use Case, Activity, Sequence, ERD tổng quát và ERD chi tiết. Các sơ đồ lần lượt mô tả phạm vi chức năng, luồng nghiệp vụ, thứ tự tương tác và cấu trúc dữ liệu của hệ thống.

2.1. Use Case Diagram

Use Case Diagram xác định ranh giới chức năng theo ba tác nhân Buyer, Seller và Admin. Nguồn thiết kế chỉ có UC-01 Buyer, UC-02 Seller và UC-03 Admin; Device/QC là thành phần hỗ trợ luồng Seller, không tạo thành UC-04 độc lập.

Sơ đồ cần tổng hợp ba tác nhân Buyer, Seller và Admin; phạm vi QC được đặt trong luồng Seller, không tách thành tác nhân độc lập. Vị trí chèn thủ công được đánh dấu tại [Hình 2.1](#).

*Nguồn diagram: Documents_Refactor/Custom_Keyboard_Use_Cases_Refactor.md;
KTPMUD_diagram/16-18_UseCase_*.drawio*



Hình 2.1. Sơ đồ Use Case tổng hợp của hệ thống Custom Keyboard Builder

2.2. Activity Diagram

Activity Diagram mô hình hóa các bước xử lý và nhánh quyết định của sáu nhóm hoạt động: tài khoản, tạo build/request, Seller xử lý request, Admin quản trị, chat và kiểm tra Device/QC.

Sơ đồ cần tổng hợp các nhánh hoạt động về tài khoản, Buyer, Seller, Admin, chat và kiểm tra Device/QC. Vị trí chèn thủ công được đánh dấu tại [Hình 2.2](#).

Nguồn diagram: *Documents_Refactor/Custom_Keyboard_Activity_Diagrams_Refactor.md*



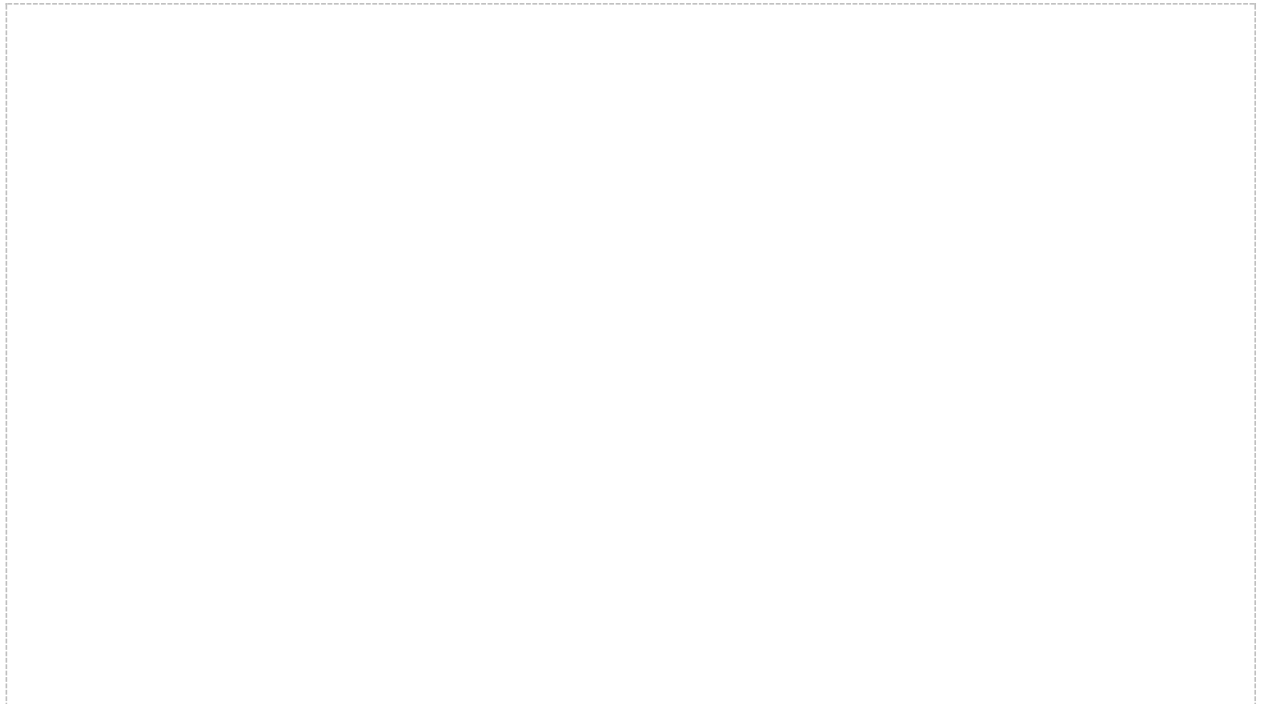
Hình 2.2. Sơ đồ Activity tổng hợp của các luồng nghiệp vụ chính

2.3. Sequence Diagram

Sequence Diagram mô tả thứ tự thông điệp giữa người dùng, View/ViewModel, Service, Repository, SQL Server và thành phần realtime. Sáu luồng lỗi bao phủ tài khoản, build request, Seller/QC, quản trị, duyệt đơn Seller và chat.

Sơ đồ cần tổng hợp trình tự giữa View/ViewModel, Service, Repository, SQL Server và thành phần realtime trong sáu luồng lỗi. Vị trí chèn thủ công được đánh dấu tại [Hình 2.3](#).

Nguồn diagram: Documents_Refactor/Custom_Keyboard_Sequence_Diagrams_6_Core.md



Hình 2.3. Sơ đồ Sequence tổng hợp của các luồng tương tác chính

2.4. Entity-Relationship Diagram (ERD)

ERD chuẩn được mô tả bằng DBML trong Documents_Refactor. Mô hình gồm 21 bảng nghiệp vụ, 21 khóa chính đều tên id và 33 khóa ngoại; dbo.schema_migrations là bảng hạ tầng nên không được tính vào ERD nghiệp vụ.

Các nhóm bảng được tổng hợp tại [Bảng 2.1](#).

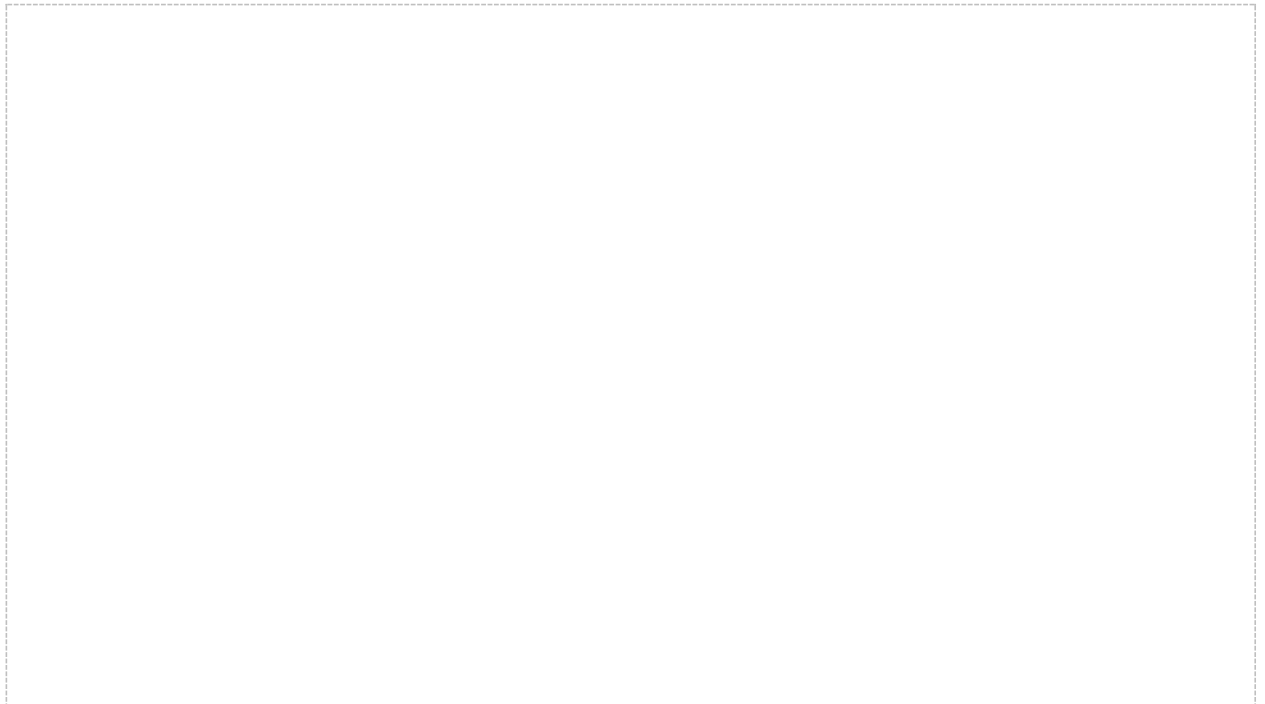
Bảng 2.1. Nhóm bảng trong cơ sở dữ liệu

Nhóm	Bảng	Mục đích
Tài khoản	roles, users, seller_profiles, seller_applications	Role, tài khoản, hồ sơ Seller và quy trình nâng cấp Buyer thành Seller.
Catalog	brands, layouts, keyboard_kits, switches, keycap_sets, stabilizers, accessories	Danh mục kit-based và thông tin tương thích cơ bản.
Build/Request	builds, build_items, build_mods, build_requests	Cấu hình, giá snapshot, mod note và yêu cầu gửi Seller.
Device/QC	devices, device_test_sessions, device_key_test_results	Trạm QC, phiên kiểm tra và kết quả từng phím.
Hỗ trợ	audit_log, chat_conversations, chat_messages	Truy vết và trao đổi theo role.

2.4.1. Sơ đồ ERD tổng quát

Sơ đồ cần thể hiện năm vùng dữ liệu: tài khoản, danh mục, build/request, Device/QC và audit/chat cùng các quan hệ chính. Vị trí chèn thủ công được đánh dấu tại [Hình 2.4](#).

Nguồn diagram: Documents_Refactor/Custom_Keyboard_ERD_Realistic_Kit_Shop_Proposal.dbml



Hình 2.4. Sơ đồ ERD tổng quát của hệ thống Custom Keyboard Builder

2.4.2. Sơ đồ ERD chi tiết

Sơ đồ cần hiển thị cột, kiểu dữ liệu, khóa chính id, khóa ngoại, UNIQUE, CHECK và hai filtered/unique index có ý nghĩa nghiệp vụ. Vị trí chèn thủ công được đánh dấu tại [Hình 2.5](#).

Nguồn diagram: Documents_Refactor/Custom_Keyboard_ERD_Realistic_Kit_Shop_Proposal.dbml;
Database/SqlServer/CreateSchema_Refactor.sql



Hình 2.5. Sơ đồ ERD chi tiết gồm 21 bảng và 33 khóa ngoại

2.5. Đối chiếu diagram với thành phần triển khai

Nguồn của năm sơ đồ thuộc bốn nhóm ký pháp được đối chiếu tại [Bảng 2.2](#).

Bảng 2.2. Đối chiếu bốn loại diagram với artifact nguồn

Nhóm ký pháp	Artifact nguồn	Phạm vi phản ánh
Use Case	Custom_Keyboard_Use_Cases_Refactor.md; Draw.io 16-18	Tác nhân và chức năng theo role; UC-02 bao gồm QC.
Activity	Custom_Keyboard_Activity_Diagrams_Refactor.md	Nhánh quyết định của sáu nhóm hoạt động.
Sequence	Custom_Keyboard_Sequence_Diagrams_6_Core.md	Tương tác theo thời gian của sáu luồng lỗi.
ERD	Custom_Keyboard_ERD_Realistic_Kit_Shop_Proposal.dbml	Một sơ đồ tổng quát và một sơ đồ chi tiết cho 21 bảng, 33 FK.

2.6. Kết luận chương

Chương 2 đã mô hình hóa phạm vi chức năng, luồng nghiệp vụ, thứ tự tương tác và cấu trúc dữ liệu thông qua năm sơ đồ thuộc bốn nhóm ký pháp. ERD chi tiết là cơ sở trực tiếp để triển khai 21 bảng SQL Server; các sơ đồ còn lại định hướng việc hiện thực giao diện và nghiệp vụ được trình bày ở Chương 3.

CHƯƠNG 3. TRIỂN KHAI VÀ KIỂM THỬ

Chương này trình bày DDL của từng bảng theo ERD, giao diện và mã nguồn tiêu biểu, kết quả build/xác minh cùng đánh giá mức độ hoàn thành. Tất cả đoạn code được đưa dưới dạng văn bản trong bảng hai hàng, không sử dụng ảnh chụp code.

3.1. Triển khai cơ sở dữ liệu SQL Server

CreateSchema_Refactor.sql được dùng cho môi trường clean/disposable và có hành vi xóa, tạo lại 21 bảng. Database có dữ liệu phải dùng chuỗi preflight - migration PK-to-id - postflight trên bản restore rehearsal; không được chạy clean schema trực tiếp.

Tại thời điểm audit ngày 15/07/2026, source code, DBML, schema và repository đã thống nhất khóa chính id, nhưng runtime database vẫn dùng 21 tên PK legacy. Vì vậy migration được xem là đã chuẩn bị nhưng chưa được xác nhận triển khai trên runtime.

Thứ tự phụ thuộc của các nhóm bảng được tóm tắt tại [Bảng 3.1](#).

Bảng 3.1. Thứ tự triển khai 21 bảng theo nhóm phụ thuộc

Thứ tự	Nhóm	Bảng	Số bảng
1	Tài khoản	roles, users, seller_profiles, seller_applications	4
2	Catalog	brands, layouts, keyboard_kits, switches, keycap_sets, stabilizers, accessories	7
3	Build/Request	builds, build_items, build_mods, build_requests	4
4	Device/QC	devices, device_test_sessions, device_key_test_results	3
5	Audit/Chat	audit_log, chat_conversations, chat_messages	3

3.1.1. Nhóm tài khoản và phân quyền

Mã SQL 3.1 - Tạo bảng roles

```
CREATE TABLE roles (  
  id INT IDENTITY(1,1) PRIMARY KEY,  
  role_name VARCHAR(50) NOT NULL UNIQUE,  
  permissions VARCHAR(MAX) NULL  
);
```

Mã SQL 3.2 - Tạo bảng users

```
CREATE TABLE users (  
  id INT IDENTITY(1,1) PRIMARY KEY,  
  role_id INT NOT NULL,  
  username VARCHAR(100) NOT NULL UNIQUE,  
  email VARCHAR(255) NOT NULL UNIQUE,  
  phone VARCHAR(30) NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  is_active BIT NOT NULL DEFAULT 1,  
  CONSTRAINT FK_users_roles FOREIGN KEY (role_id) REFERENCES roles(id)  
);
```

Mã SQL 3.3 - Tạo bảng seller_profiles

```
CREATE TABLE seller_profiles (  
  id INT IDENTITY(1,1) PRIMARY KEY,  
  user_id INT NOT NULL UNIQUE,  
  shop_name VARCHAR(255) NOT NULL,  
  phone VARCHAR(30) NOT NULL,  
  address VARCHAR(500) NOT NULL,  
  is_verified BIT NOT NULL DEFAULT 0,  
  verified_at DATETIME2 NULL,  
  CONSTRAINT FK_seller_profiles_user FOREIGN KEY (user_id) REFERENCES users(id)  
);
```

Mã SQL 3.4 - Tạo bảng seller_applications

```
CREATE TABLE seller_applications (  
  id INT IDENTITY(1,1) PRIMARY KEY,  
  buyer_user_id INT NOT NULL,  
  shop_name VARCHAR(255) NOT NULL,  
  phone VARCHAR(30) NOT NULL,  
  address VARCHAR(500) NOT NULL,  
  note VARCHAR(500) NULL,  
  status VARCHAR(50) NOT NULL DEFAULT 'Pending',  
  review_note VARCHAR(500) NULL,  
  created_at DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),  
  reviewed_at DATETIME2 NULL,  
  reviewed_by INT NULL,  
  CONSTRAINT FK_seller_applications_buyer FOREIGN KEY (buyer_user_id) REFERENCES users(id),  
  CONSTRAINT FK_seller_applications_reviewer FOREIGN KEY (reviewed_by) REFERENCES users(id),  
  CONSTRAINT CK_seller_applications_status CHECK (status IN ('Pending', 'Approved', 'Rejected'))  
);  
  
CREATE UNIQUE INDEX UX_seller_applications_pending_buyer  
ON seller_applications(buyer_user_id)  
WHERE status = 'Pending';
```

3.1.2. Nhóm danh mục linh kiện

Mã SQL 3.5 - Tạo bảng brands

```
CREATE TABLE brands (  
  id INT IDENTITY(1,1) PRIMARY KEY,  
  brand_name VARCHAR(255) NOT NULL UNIQUE,  
  country VARCHAR(100) NULL  
);
```

Mã SQL 3.6 - Tạo bảng layouts

```
CREATE TABLE layouts (  
  id VARCHAR(50) PRIMARY KEY,  
  layout_name VARCHAR(100) NOT NULL,  
  form_factor VARCHAR(100) NOT NULL,  
  key_count INT NOT NULL  
);
```

Mã SQL 3.7 - Tạo bảng keyboard_kits

```
CREATE TABLE keyboard_kits (  
    id VARCHAR(50) PRIMARY KEY,  
    brand_id INT NOT NULL,  
    layout_id VARCHAR(50) NOT NULL,  
    kit_name VARCHAR(255) NOT NULL,  
    pcb_technology VARCHAR(50) NOT NULL,  
    switch_mount VARCHAR(100) NOT NULL,  
    required_switch_quantity INT NOT NULL,  
    included_parts VARCHAR(500) NULL,  
    price_usd DECIMAL(10,2) NOT NULL,  
    is_available BIT NOT NULL DEFAULT 1,  
    CONSTRAINT FK_keyboard_kits_brands FOREIGN KEY (brand_id) REFERENCES brands(id),  
    CONSTRAINT FK_keyboard_kits_layouts FOREIGN KEY (layout_id) REFERENCES layouts(id),  
    CONSTRAINT CK_keyboard_kits_price CHECK (price_usd >= 0),  
    CONSTRAINT CK_keyboard_kits_switch_qty CHECK (required_switch_quantity > 0)  
);
```

Mã SQL 3.8 - Tạo bảng switches

```
CREATE TABLE switches (  
    id VARCHAR(50) PRIMARY KEY,  
    brand_id INT NOT NULL,  
    switch_name VARCHAR(255) NOT NULL,  
    switch_technology VARCHAR(50) NOT NULL,  
    mount_type VARCHAR(100) NOT NULL,  
    switch_type VARCHAR(100) NULL,  
    actuation_force_g INT NULL,  
    price_usd DECIMAL(10,2) NOT NULL,  
    is_available BIT NOT NULL DEFAULT 1,  
    CONSTRAINT FK_switches_brands FOREIGN KEY (brand_id) REFERENCES brands(id),  
    CONSTRAINT CK_switches_price CHECK (price_usd >= 0)  
);
```

Mã SQL 3.9 - Tạo bảng keycap_sets

```
CREATE TABLE keycap_sets (  
    id VARCHAR(50) PRIMARY KEY,  
    brand_id INT NOT NULL,  
    keycap_name VARCHAR(255) NOT NULL,  
    supported_form_factor VARCHAR(255) NOT NULL,  
    profile VARCHAR(100) NULL,  
    material VARCHAR(100) NULL,  
    price_usd DECIMAL(10,2) NOT NULL,  
    is_available BIT NOT NULL DEFAULT 1,  
    CONSTRAINT FK_keycap_sets_brands FOREIGN KEY (brand_id) REFERENCES brands(id),  
    CONSTRAINT CK_keycap_sets_price CHECK (price_usd >= 0)  
);
```

Mã SQL 3.10 - Tạo bảng stabilizers

```
CREATE TABLE stabilizers (  
    id VARCHAR(50) PRIMARY KEY,  
    brand_id INT NOT NULL,  
    stab_name VARCHAR(255) NOT NULL,  
    supported_layouts VARCHAR(255) NOT NULL,  
    price_usd DECIMAL(10,2) NOT NULL,  
    is_available BIT NOT NULL DEFAULT 1,  
    CONSTRAINT FK_stabilizers_brands FOREIGN KEY (brand_id) REFERENCES brands(id),  
    CONSTRAINT CK_stabilizers_price CHECK (price_usd >= 0)  
);
```

Mã SQL 3.11 - Tạo bảng accessories

```
CREATE TABLE accessories (  
  id VARCHAR(50) PRIMARY KEY,  
  accessory_type VARCHAR(100) NOT NULL,  
  accessory_name VARCHAR(255) NOT NULL,  
  target_component VARCHAR(100) NULL,  
  price_usd DECIMAL(10,2) NOT NULL,  
  is_available BIT NOT NULL DEFAULT 1,  
  CONSTRAINT CK_accessories_price CHECK (price_usd >= 0)  
);
```

3.1.3. Nhóm build và request

Mã SQL 3.12 - Tạo bảng builds

```
CREATE TABLE builds (  
  id VARCHAR(50) PRIMARY KEY,  
  buyer_id INT NOT NULL,  
  kit_id VARCHAR(50) NOT NULL,  
  name VARCHAR(255) NOT NULL,  
  notes VARCHAR(500) NULL,  
  noise_requirement VARCHAR(20) NOT NULL DEFAULT 'Normal',  
  status VARCHAR(50) NOT NULL,  
  total_cost_snapshot DECIMAL(10,2) NOT NULL,  
  created_at DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),  
  updated_at DATETIME2 NULL,  
  CONSTRAINT FK_builds_buyer FOREIGN KEY (buyer_id) REFERENCES users(id),  
  CONSTRAINT FK_builds_kit FOREIGN KEY (kit_id) REFERENCES keyboard_kits(id),  
  CONSTRAINT CK_builds_noise_requirement CHECK (noise_requirement IN ('Normal', 'Quiet', 'Silent')),  
  CONSTRAINT CK_builds_status CHECK (status IN ('Draft', 'Saved', 'Requested', 'Archived')),  
  CONSTRAINT CK_builds_total CHECK (total_cost_snapshot >= 0)  
);
```

Mã SQL 3.13 - Tạo bảng build_items

```
CREATE TABLE build_items (  
  id INT IDENTITY(1,1) PRIMARY KEY,  
  build_id VARCHAR(50) NOT NULL,  
  switch_id VARCHAR(50) NULL,  
  keycap_id VARCHAR(50) NULL,  
  stab_id VARCHAR(50) NULL,  
  accessory_id VARCHAR(50) NULL,  
  quantity INT NOT NULL,  
  unit_price_snapshot DECIMAL(10,2) NOT NULL,  
  notes VARCHAR(500) NULL,  
  CONSTRAINT FK_build_items_build FOREIGN KEY (build_id) REFERENCES builds(id),  
  CONSTRAINT FK_build_items_switch FOREIGN KEY (switch_id) REFERENCES switches(id),  
  CONSTRAINT FK_build_items_keycap FOREIGN KEY (keycap_id) REFERENCES keycap_sets(id),  
  CONSTRAINT FK_build_items_stab FOREIGN KEY (stab_id) REFERENCES stabilizers(id),  
  CONSTRAINT FK_build_items_accessory FOREIGN KEY (accessory_id) REFERENCES accessories(id),  
  CONSTRAINT CK_build_items_quantity CHECK (quantity > 0),  
  CONSTRAINT CK_build_items_unit_price CHECK (unit_price_snapshot >= 0),  
  
  CONSTRAINT CK_build_items_exactly_one_fk CHECK (  
    (CASE WHEN switch_id IS NULL THEN 0 ELSE 1 END) +  
    (CASE WHEN keycap_id IS NULL THEN 0 ELSE 1 END) +  
    (CASE WHEN stab_id IS NULL THEN 0 ELSE 1 END) +  
    (CASE WHEN accessory_id IS NULL THEN 0 ELSE 1 END) = 1  
  )  
);
```

Mã SQL 3.14 - Tạo bảng build_mods

```
CREATE TABLE build_mods (  
  id INT IDENTITY(1,1) PRIMARY KEY,  
  build_id VARCHAR(50) NOT NULL,  
  mod_type VARCHAR(100) NOT NULL,  
  target_component VARCHAR(100) NOT NULL,  
  notes VARCHAR(500) NULL,  
  CONSTRAINT FK_build_mods_build FOREIGN KEY (build_id) REFERENCES builds(id)  
);
```


Mã SQL 3.15 - Tạo bảng build_requests

```
CREATE TABLE build_requests (
  id VARCHAR(50) PRIMARY KEY,
  build_id VARCHAR(50) NOT NULL,
  seller_user_id INT NOT NULL,
  request_payload_json NVARCHAR(MAX) NOT NULL,
  status VARCHAR(50) NOT NULL,
  note VARCHAR(500) NULL,
  requested_at DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
  accepted_at DATETIME2 NULL,
  completed_at DATETIME2 NULL,
  updated_at DATETIME2 NULL,
  CONSTRAINT FK_build_requests_build FOREIGN KEY (build_id) REFERENCES builds(id),
  CONSTRAINT FK_build_requests_seller FOREIGN KEY (seller_user_id) REFERENCES users(id),
  CONSTRAINT CK_build_requests_status CHECK (status IN ('Pending', 'Accepted', 'In_progress', 'Completed',
'Cancelled'))
);
```

3.1.4. Nhóm thiết bị và kiểm tra QC

Mã SQL 3.16 - Tạo bảng devices

```
CREATE TABLE devices (
  id VARCHAR(50) PRIMARY KEY,
  seller_user_id INT NOT NULL,
  device_name NVARCHAR(100) NOT NULL,
  device_type VARCHAR(50) NOT NULL,
  is_active BIT NOT NULL DEFAULT 1,
  last_seen_at DATETIME2 NULL,
  created_at DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
  CONSTRAINT FK_devices_seller FOREIGN KEY (seller_user_id) REFERENCES users(id),
  CONSTRAINT CK_devices_type CHECK (device_type IN
('QC_STATION', 'KEY_SIGNAL_TESTER', 'LATENCY_TESTER', 'NOISE_SENSOR'))
);
```

Mã SQL 3.17 - Tạo bảng device_test_sessions

```
CREATE TABLE device_test_sessions (
  id VARCHAR(50) PRIMARY KEY,
  request_id VARCHAR(50) NOT NULL,
  device_id VARCHAR(50) NOT NULL,
  seller_user_id INT NOT NULL,
  switch_technology VARCHAR(50) NOT NULL,
  noise_requirement VARCHAR(20) NOT NULL DEFAULT 'Normal',
  total_keys INT NOT NULL,
  tested_keys INT NOT NULL DEFAULT 0,
  passed_keys INT NOT NULL DEFAULT 0,
  warning_keys INT NOT NULL DEFAULT 0,
  failed_keys INT NOT NULL DEFAULT 0,
  average_latency_ms DECIMAL(8,2) NULL,
  max_latency_ms DECIMAL(8,2) NULL,
  average_noise_db DECIMAL(8,2) NULL,
  max_noise_db DECIMAL(8,2) NULL,
  status VARCHAR(20) NOT NULL,
  started_at DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
  completed_at DATETIME2 NULL,
  CONSTRAINT FK_dts_request FOREIGN KEY (request_id) REFERENCES build_requests(id),
  CONSTRAINT FK_dts_device FOREIGN KEY (device_id) REFERENCES devices(id),
  CONSTRAINT FK_dts_seller FOREIGN KEY (seller_user_id) REFERENCES users(id),
  CONSTRAINT CK_dts_noise_requirement CHECK (noise_requirement IN ('Normal', 'Quiet', 'Silent')),
  CONSTRAINT CK_dts_status CHECK (status IN ('Running', 'Passed', 'Warning', 'Failed'))
);
```

Mã SQL 3.18 - Tạo bảng device_key_test_results

```
CREATE TABLE device_key_test_results (
  id BIGINT IDENTITY(1,1) PRIMARY KEY,
  session_id VARCHAR(50) NOT NULL,
  request_id VARCHAR(50) NOT NULL,
  device_id VARCHAR(50) NOT NULL,
  key_code VARCHAR(30) NOT NULL,
  expected_key VARCHAR(30) NOT NULL,
  received_key VARCHAR(30) NULL,
  press_signal_detected BIT NOT NULL,
  latency_ms DECIMAL(8,2) NULL,
  press_event_count INT NOT NULL,
  bounce_count INT NULL,
  release_signal_detected BIT NOT NULL,
  hold_duration_ms INT NULL,
  is_stuck BIT NOT NULL,
  noise_db DECIMAL(8,2) NULL,
  switch_technology VARCHAR(50) NOT NULL,
  result VARCHAR(20) NOT NULL,
  failure_type VARCHAR(30) NULL,
  failure_reason NVARCHAR(255) NULL,
  recorded_at DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
  CONSTRAINT FK_dktr_session FOREIGN KEY (session_id) REFERENCES device_test_sessions(id),
  CONSTRAINT FK_dktr_request FOREIGN KEY (request_id) REFERENCES build_requests(id),
  CONSTRAINT FK_dktr_device FOREIGN KEY (device_id) REFERENCES devices(id),
  CONSTRAINT CK_dktr_result CHECK (result IN ('Pass', 'Warning', 'Fail')),
  CONSTRAINT CK_dktr_failure_type CHECK (
    failure_type IS NULL
    OR failure_type IN ('NoSignal', 'WrongKey', 'Chatter', 'StuckKey', 'HighLatency', 'TooNoisy')
  )
);

CREATE UNIQUE INDEX UX_dktr_session_key_code
ON device_key_test_results(session_id, key_code);
```

3.1.5. Nhóm audit và chat

Mã SQL 3.19 - Tạo bảng audit_log

```
CREATE TABLE audit_log (
  id INT IDENTITY(1,1) PRIMARY KEY,
  user_id INT NOT NULL,
  table_name VARCHAR(100) NOT NULL,
  record_id VARCHAR(100) NOT NULL,
  action VARCHAR(100) NOT NULL,
  old_value_json NVARCHAR(MAX) NULL,
  new_value_json NVARCHAR(MAX) NULL,
  changed_at DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
  CONSTRAINT FK_audit_log_user FOREIGN KEY (user_id) REFERENCES users(id)
);
```

Mã SQL 3.20 - Tạo bảng chat_conversations

```
CREATE TABLE chat_conversations (
  id VARCHAR(50) PRIMARY KEY,
  seller_user_id INT NOT NULL,
  buyer_id INT NULL,
  admin_user_id INT NULL,
  build_request_id VARCHAR(50) NULL,
  created_at DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
  updated_at DATETIME2 NULL,
  CONSTRAINT FK_chat_conversations_seller FOREIGN KEY (seller_user_id) REFERENCES users(id),
  CONSTRAINT FK_chat_conversations_buyer FOREIGN KEY (buyer_id) REFERENCES users(id),
  CONSTRAINT FK_chat_conversations_admin FOREIGN KEY (admin_user_id) REFERENCES users(id),
  CONSTRAINT FK_chat_conversations_request FOREIGN KEY (build_request_id) REFERENCES build_requests(id),

  CONSTRAINT CK_chat_conversations_participant CHECK (
    (CASE WHEN buyer_id IS NULL THEN 0 ELSE 1 END) +
    (CASE WHEN admin_user_id IS NULL THEN 0 ELSE 1 END) = 1
  )
);
```

Mã SQL 3.21 - Tạo bảng chat_messages

```
CREATE TABLE chat_messages (  
    id VARCHAR(50) PRIMARY KEY,  
    conversation_id VARCHAR(50) NOT NULL,  
    sender_user_id INT NOT NULL,  
    message_text NVARCHAR(MAX) NOT NULL,  
    sent_at DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),  
    CONSTRAINT FK_chat_messages_conversation FOREIGN KEY (conversation_id) REFERENCES chat_conversations(id),  
    CONSTRAINT FK_chat_messages_sender FOREIGN KEY (sender_user_id) REFERENCES users(id)  
);
```

3.2. Triển khai giao diện tiêu biểu

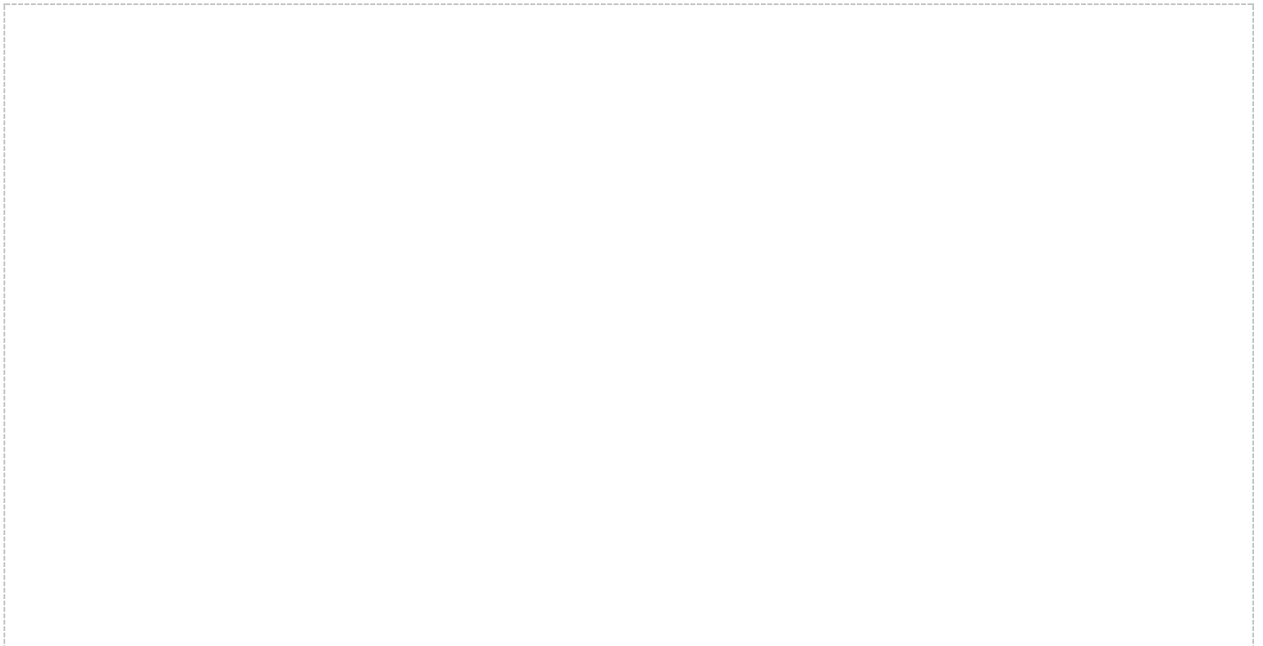
Mối liên hệ giữa giao diện và mã nguồn được tổng hợp tại [Bảng 3.2](#).

Bảng 3.2. Đối chiếu giao diện với mã nguồn

Giao diện	View	ViewModel/Service	Nội dung minh họa
Đăng nhập	Views/LoginView.xaml	LoginViewModel; AccountService	Nhập thông tin, xác thực, báo lỗi và điều hướng theo role.
Đăng ký	Views/RegisterView.xaml	RegisterViewModel; AccountService	Đăng ký Buyer và kiểm tra dữ liệu đầu vào.
Buyer Dashboard	Views/ BuyerDashboard View.xaml	BuyerDashboardViewModel; BuildService; RequestService	Cấu hình build, tính giá, validation và gửi request.
Seller Dashboard	Views/ SellerDashboard View.xaml	SellerDashboardViewModel; RequestService; DeviceService	Xử lý request, trạng thái và QC mô phỏng.
Admin Dashboard	Views/ AdminDashboard View.xaml	AdminDashboardViewModel; AdminService; StatsService	KPI, user, Seller, catalog, application và audit.
Chat	Views/ChatView.xaml	ChatViewModel; ChatService	Hội thoại Buyer-Seller và Admin-Seller có kiểm tra quyền.
User Menu	Views/ UserMenuView.xaml	MainShellViewModel	Hồ sơ, ngôn ngữ và đăng xuất.

Vùng hình dành cho ảnh chụp màn hình đăng nhập, thông báo trạng thái và điều hướng tạo tài khoản. Vị trí chèn thủ công được đánh dấu tại [Hình 3.1](#).

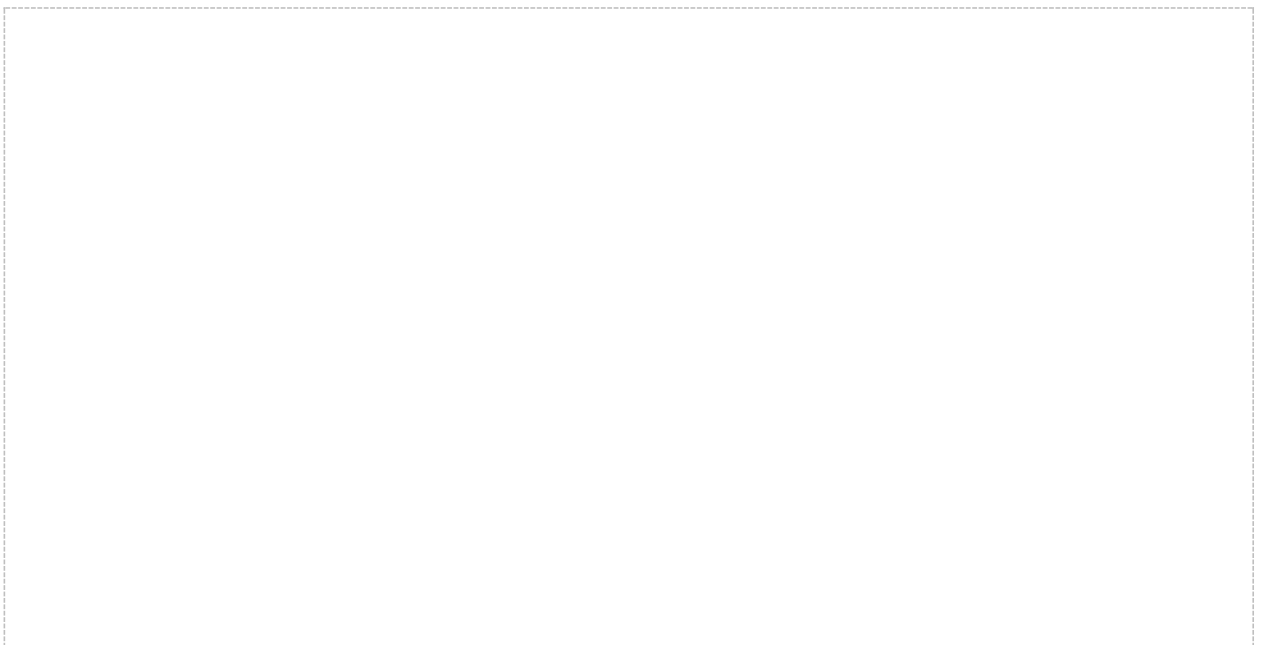
Nguồn diagram: Views/LoginView.xaml



Hình 3.1. Giao diện đăng nhập của ứng dụng

Vùng hình dành cho ảnh chụp biểu mẫu đăng ký, kiểm tra dữ liệu đầu vào và phản hồi kết quả. Vị trí chèn thủ công được đánh dấu tại [Hình 3.2](#).

Nguồn diagram: Views/RegisterView.xaml



Hình 3.2. Giao diện đăng ký tài khoản Buyer

Mã nguồn 3.22 - Form đăng nhập bằng XAML

```
<TextBlock Text="{loc:Tr Login_EmailOrUsername}"
           Style="{StaticResource FieldLabel}"/>
<TextBox AutomationProperties.AutomationId="LoginEmailOrUsernameTextBox"
          Text="{Binding EmailOrUsername, UpdateSourceTrigger=PropertyChanged}"/>

<TextBlock Text="{loc:Tr Common_Password}"
           Style="{StaticResource FieldLabel}"/>
<PasswordBox AutomationProperties.AutomationId="LoginPasswordBox"
              behaviors:PasswordBoxBinding.Attach="True"
              behaviors:PasswordBoxBinding.BoundPassword="{Binding Password, Mode=TwoWay}"/>

<TextBlock Text="{Binding ErrorMessage}"
           Foreground="{StaticResource DangerBrush}"/>
<Button Content="{loc:Tr Login_SignIn}"
        Command="{Binding LoginCommand}"
        Style="{StaticResource PrimaryButton}"/>
```

Mã nguồn 3.23 - ViewModel xử lý đăng nhập

```
private async Task LoginAsync()
{
    ErrorMessage = string.Empty;
    StatusMessage = string.Empty;

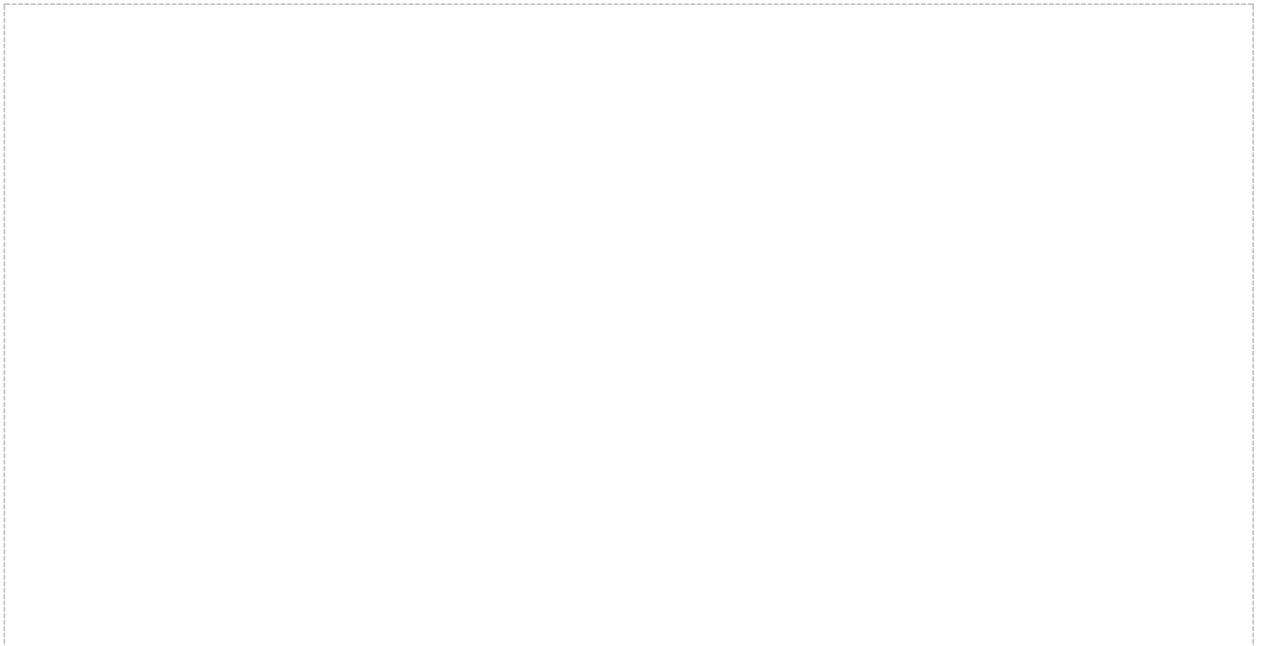
    var result = await _accountService.LoginAsync(
        EmailOrUsername,
        Password);

    if (result.Succeeded && result.User is not null)
    {
        Password = string.Empty;
        _loginSucceeded(result.User);
        return;
    }

    ErrorMessage = result.Message;
}
```

Vùng hình dành cho ảnh chụp configurator kit-based, tổng giá snapshot, validation và thao tác gửi request. Vị trí chèn thủ công được đánh dấu tại [Hình 3.3](#).

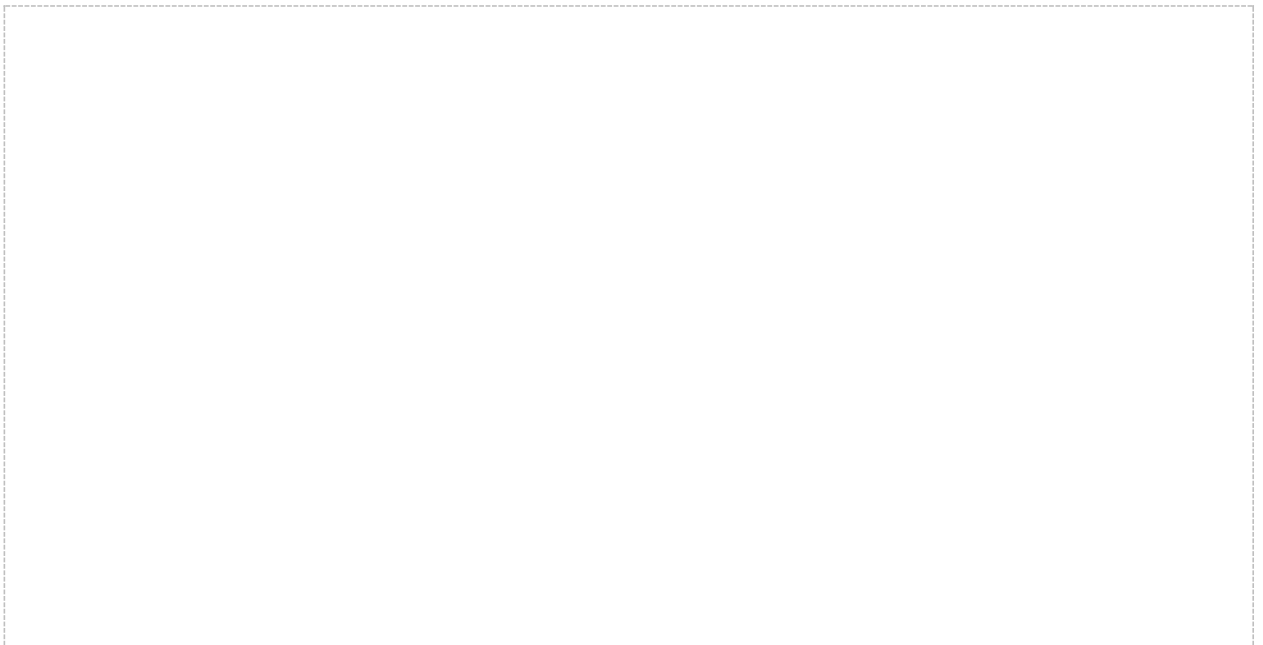
Nguồn diagram: Views/BuyerDashboardView.xaml



Hình 3.3. Giao diện Buyer Dashboard và khu vực cấu hình build

Vùng hình dành cho ảnh chụp danh sách request, chuyển trạng thái và khu vực thực hiện kiểm tra QC. Vị trí chèn thủ công được đánh dấu tại [Hình 3.4](#).

Nguồn diagram: Views/SellerDashboardView.xaml



Hình 3.4. Giao diện Seller Dashboard và xử lý yêu cầu lắp ráp

Vùng hình dành cho ảnh chụp KPI, quản lý tài khoản, Seller, catalog, đơn đăng ký và audit log. Vị trí chèn thủ công được đánh dấu tại [Hình 3.5](#).

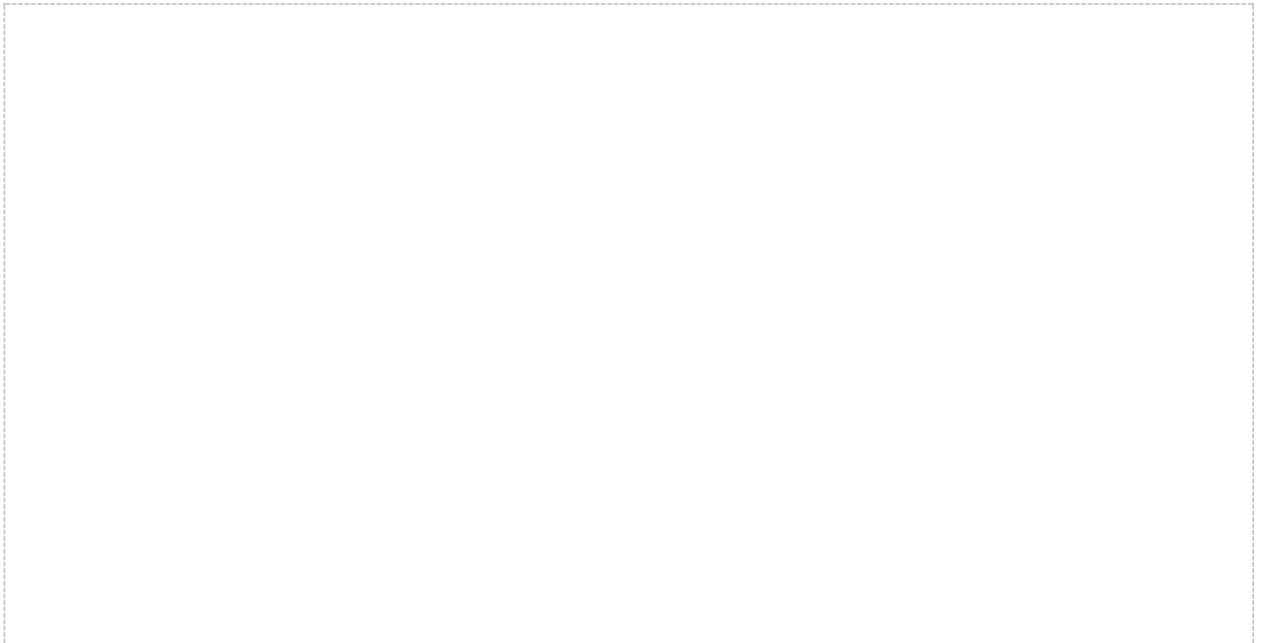
Nguồn diagram: Views/AdminDashboardView.xaml



Hình 3.5. Giao diện Admin Dashboard và quản trị hệ thống

Vùng hình dành cho ảnh chụp danh sách hội thoại và nội dung chat Buyer-Seller hoặc Admin-Seller. Vị trí chèn thủ công được đánh dấu tại [Hình 3.6](#).

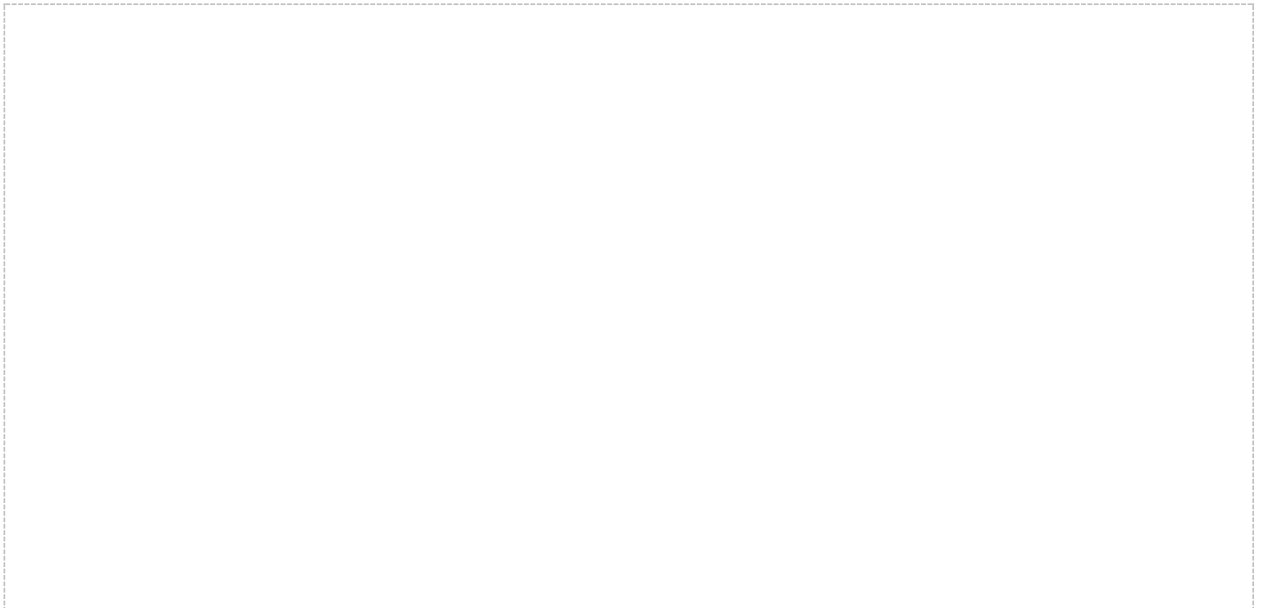
Nguồn diagram: Views/ChatView.xaml



Hình 3.6. Giao diện trao đổi tin nhắn theo vai trò

Vùng hình dành cho ảnh chụp thông tin tài khoản, lựa chọn ngôn ngữ và thao tác đăng xuất. Vị trí chèn thủ công được đánh dấu tại [Hình 3.7](#).

Nguồn diagram: *Views/UserMenuView.xaml*



Hình 3.7. Giao diện hồ sơ và menu người dùng

Mã nguồn 3.24 - Khu vực cấu hình build bằng XAML

```
<TextBlock Text="{loc:Tr Buyer_KitLabel}"
    Style="{StaticResource FieldLabel}"/>
<ComboBox ItemsSource="{Binding Kits}"
    SelectedItem="{Binding SelectedKit}"
    DisplayMemberPath="KitName"/>

<TextBlock Text="{loc:Tr Buyer_SwitchLabel}"
    Style="{StaticResource FieldLabel}"/>
<ComboBox ItemsSource="{Binding CompatibleSwitches}"
    SelectedItem="{Binding SelectedSwitch}"
    DisplayMemberPath="SwitchName"/>

<TextBlock Text="{Binding TotalPreview,
    StringFormat={}{0:N2} USD}"/>
<Button Content="{loc:Tr Buyer_SaveBuild}"
    Command="{Binding SaveBuildCommand}"/>
<Button Content="{loc:Tr Buyer_SendRequest}"
    Command="{Binding SendRequestCommand}"/>
```


Mã nguồn 3.25 - Lưu build và gửi request trong ViewModel

```
private async Task SaveBuildAsync()
{
    var build = CreateBuildFromCurrentSelection();
    var saved = await _buildService.SaveBuildAsync(build);
    _editingBuildId = saved.BuildId;
    await RefreshBuildsAsync();
    StatusMessage = TrFormat(
        "Buyer_BuildSaved",
        saved.Name,
        saved.TotalCostSnapshot);
}

private async Task SendRequestAsync()
{
    if (SelectedBuild is null)
        throw new InvalidOperationException(
            Tr("Buyer_SelectSavedBuildFirst"));
    if (SelectedSeller is null)
        throw new InvalidOperationException(
            Tr("Buyer_SelectSellerFirst"));

    await _requestService.SendRequestAsync(
        SelectedBuild.BuildId,
        CurrentUser.UserId,
        SelectedSeller.UserId,
        RequestNote);
    await RefreshRequestsAsync();
}
```

3.3. Kiểm thử và xác minh

Việc xác minh được thực hiện theo phạm vi không làm thay đổi dữ liệu runtime. Kết quả hiện tại được trình bày tại [Bảng 3.3](#); các tuyên bố kiểm thử lịch sử không được dùng thay cho runner còn tồn tại trong working tree.

Bảng 3.3. Kết quả build và xác minh tại thời điểm lập báo cáo

Mã	Nội dung	Kết quả	Ghi chú
T01	dotnet build	Pass	0 warning, 0 error; Custom_keyboard.dll được tạo cho net10.0-windows.
T02	VerifyPkToId_Source.ps1	Pass	21 PK id, 33 FK, 21 rename, version/startup guard và repository scan đạt.
T03	Database runtime postflight	Chưa đạt	Runtime tại thời điểm audit vẫn dùng 21 PK legacy; migration chưa chạy.
T04	VerifyRefactor.sql trên clean-seed	Chưa chạy trong lượt lập báo cáo	Script có kiểm tra schema/data nhưng có thể ghi dữ liệu fixture; không dùng trên runtime legacy.
T05	C# integration/UI automation	Không khả dụng	Phase6Verification và WpfUiVerification không còn trong working tree hiện tại.

3.4. Đánh giá mức độ hoàn thành

Mức độ hoàn thành theo từng mục tiêu được tổng hợp tại [Bảng 3.4](#).

Bảng 3.4. Đánh giá mức độ hoàn thành theo mục tiêu

Mục tiêu	Trạng thái	Đánh giá
Phân tích thiết kế	Đạt	Có ERD, Use Case, Sequence, Activity và mapping artifact.
Giao diện và nghiệp vụ ba vai trò	Đạt ở mức mã nguồn	View/ViewModel/Service/Repository cho Buyer, Seller, Admin đã được triển khai.
Schema SQL Server	Đạt ở source	DBML/schema/migration/verifier đã đồng bộ 21 PK id và 33 FK.
Vận hành trên runtime database	Chưa đạt	Cần backup, restore rehearsal, migration, postflight và smoke test.
QC phần cứng	Ngoài phạm vi	QC hiện dùng DeviceSimulator; không có ESP32/phần cứng thật.
Chat realtime SignalR	Chưa triển khai	Chat đã lưu DB; SignalR là phần mở rộng tùy chọn.

3.5. Kết luận chương

Chương 3 đã trình bày 21 khối DDL, bảy vị trí chèn ảnh giao diện, bốn đoạn mã giao diện và kết quả xác minh. Việc nghiệm thu vận hành còn phụ thuộc vào quá trình migration runtime và smoke test sau migration.

KẾT LUẬN

1. Mức độ hoàn thành nhiệm vụ

Các nhiệm vụ phân tích, thiết kế và triển khai mã nguồn cốt lõi đã được hoàn thành phần lớn: ứng dụng WPF theo MVVM, phân quyền Buyer/Seller/Admin, cấu hình build kit-based, request state machine, Seller application, chat lưu database, audit, analytics và QC mô phỏng đều có thành phần triển khai tương ứng.

Tuy nhiên, nhiệm vụ chưa thể được coi là hoàn thành ở mức vận hành cuối cùng. Runtime database được audit ngày 15/07/2026 vẫn dùng schema PK legacy, do đó cần hoàn tất backup, restore rehearsal, migration, postflight và smoke test trước khi nghiệm thu.

2. Mức độ phù hợp với mục đích đề tài

Thiết kế hiện tại phù hợp với mục đích số hóa quy trình cấu hình và đặt lắp ráp bàn phím tùy chỉnh. Mô hình kit-based giảm độ phức tạp so với việc chọn case/PCB/plate rời; snapshot pricing, validation và phân quyền hỗ trợ tính đúng đắn của dữ liệu; SQL Server tạo nền tảng lưu trữ tập trung và truy vết.

3. Hạn chế còn tồn tại

- Runtime database chưa áp dụng migration PK-to-id và chưa có bằng chứng smoke test sau migration.
- Phase6Verification và WpfUiVerification không còn trong working tree nên chưa thể chạy lại bộ C# integration/UI automation lịch sử.
- QC mới là mô phỏng phần mềm, chưa kết nối ESP32 hoặc trạm đo phần cứng thật.
- SignalR chat realtime chưa được triển khai; MQTT chủ yếu phục vụ thông báo request/status và telemetry tùy chọn.
- Năm cột logical enum gồm role_name và switch technology chưa có CHECK constraint đầy đủ trong schema hiện hành.

4. Hướng phát triển

- Diễn tập migration trên bản restore, chạy postflight, khôi phục runner integration/UI và ghi nhận biên bản smoke test.
- Bổ sung CHECK constraint, kiểm thử bảo mật/đầu vào/log, đồng thời hoàn thiện SignalR và cơ chế cancellation/versioning.
- Kết nối trạm QC phần cứng hoặc ESP32 khi phạm vi đề tài được mở rộng sang IoT.